Raz Standard Project Format

Canonical layout for Raz applications, libraries, workspaces, tests, benchmarks, tooling, native boundaries, and generated artifacts.

1. Canonical Full Layout

```
my-project/
|-- .github/
|-- assets/
|-- benches/
|-- bin/
|-- build/
|-- config/
|-- data/
|-- docs/
|-- examples/
|-- fuzz/
|-- generated/
|-- native/
|   |-- asm/
|   |-- include/
|   `-- src/
|-- packages/
|-- schemas/
|-- scripts/
|-- src/
|-- tests/
|-- third_party/
|-- tools/
|-- vendor/
|-- .gitignore
|-- .razfmt
|-- .razignore
|-- CHANGELOG.md
|-- LICENSE
|-- README.md
|-- raz.toml
`-- raz.lock
```

The full layout is a vocabulary, not a requirement. Small projects should only create the directories they actually need.

2. Core Directory Semantics

Path	Purpose
src/	Primary Raz implementation. Applications conventionally use src/main.rz; libraries use src/lib.rz.
bin/	Additional executable entrypoints for projects that produce multiple programs or CLI tools.
tests/	Integration, conformance, end-to-end, and system tests. Unit tests may live beside source.
benches/	Performance benchmark targets discovered by the Raz toolchain.
examples/	Small runnable examples demonstrating project APIs and language usage.
packages/	Workspace-local Raz packages for large repositories.
scripts/	Build, release, CI, packaging, generation, and developer convenience scripts.
tools/	Project-specific developer tools.
docs/	Enduring architecture, usage, design, user, and developer documentation.

Path	Purpose
assets/	Static resources, templates, fixtures, and embedded files.
generated/	Generated source, bindings, protocol code, or schema output; usually gitignored unless intentionally versioned.
native/	Optional C/C++/assembly platform and interoperability boundary.
config/	Application/runtime configuration that is separate from package metadata.
schemas/	IDL, protocol, serialization, RPC, and other machine-readable schemas.
fuzz/	Fuzz targets, corpora, dictionaries, and reproducers.
third_party/	External source trees imported directly into the repository.
vendor/	Optional vendored package dependencies for hermetic or offline builds.
build/	Compiler/linker output. Normally gitignored.

3. Source and Executable Conventions

Raz should make the common case obvious. Applications use **src/main.rz**; libraries use **src/lib.rz**. Larger modules live under src/ as normal directories.

```
src/
|-- main.rz
|-- lib.rz
|-- net/
|-- storage/
`-- runtime/
```

Multiple executables

Projects exposing more than one executable should place additional entrypoints in **bin/**. Shared implementation remains in src/.

```
bin/
|-- razd.rz
|-- razctl.rz
`-- benchmark.rz
```

A project using this layout can naturally produce executables such as **razd**, **razctl**, and **benchmark** without cluttering the primary source tree.

4. Testing and Benchmarking

```
tests/
|-- integration/
|-- conformance/
|-- fixtures/
`-- common/

benches/
|-- parser.rz
|-- networking.rz
|-- allocator.rz
`-- throughput.rz
```

The intended convention is that **raz test** discovers integration/system tests and **raz bench** discovers benchmark targets. Unit tests may stay next to the source they validate.

5. Examples and Workspace Packages

Examples should be small, runnable, and directly discoverable by the toolchain.

```
examples/
|-- hello.rz
|-- tcp-server.rz
`-- async-http.rz
```

Example invocation: **raz run --example tcp-server**.

Workspace packages

The packages/ directory gives large Raz repositories a first-class workspace model without requiring every internal component to become a separate repository.

```
packages/
|-- core/
|-- protocol/
|-- database/
|-- networking/
`-- cli/
```

Workspace membership and package relationships should be declared in **raz.toml**.

6. Native Interoperability

Native code should be optional and visibly isolated. A dedicated **native/** directory is preferable to a root include/ directory because it makes the C/C++/assembly boundary explicit.

```
native/
|-- include/
|   `-- platform.h
|-- src/
|   |-- linux.c
|   `-- windows.c
`-- asm/
```

7. Project Metadata

File	Role
raz.toml	Canonical project/package/workspace manifest. Declares metadata, targets, dependencies, features, build settings, and workspace members.
raz.lock	Resolved dependency graph for reproducible builds.
.razfmt	Project-specific formatter configuration.
.razignore	Ignore rules for Raz tooling, packaging, indexing, and related commands.
README.md	Primary project overview, setup, usage, and contributor orientation.
CHANGELOG.md	Release-facing history and meaningful changes.
LICENSE	Project licensing terms.

8. Minimal Raz Project

A normal new application should start small and add directories only when they become useful.

```
hello/
|-- src/
|   `-- main.rz
|-- tests/
|-- README.md
`-- raz.toml
```

9. Recommended raz new Modes

The project generator can expose a small set of canonical creation modes:

```
raz new hello
raz new --lib math
raz new --workspace engine
```

raz new creates an application, **--lib** creates a library rooted at src/lib.rz, and **--workspace** creates a multi-package repository centered on packages/ and workspace membership in raz.toml.

10. Recommended Standard

The most important additions to the original bin/docs/src/tools/tests structure are **examples/**, **benches/**, **packages/**, **scripts/**, **native/**, plus the **raz.toml** and **raz.lock** conventions. Together they provide a complete project model for small programs, reusable libraries, production applications, and large Raz workspaces.

Status: Proposed canonical Raz project structure.